

INSuRE Fall 2025

Project Midterm Report - 10/31/2025

Artificial Intelligence for Identifying Data Patterns in Unintended RF Emanations (v1)



Investigators:

The University of Texas at Dallas - Team 2

Adithya Chintala – Student Researcher – Adithya.Chintala@UTDallas.edu
Brigham Thornock – Student Researcher – Brigham.Thornock@UTDallas.edu
Ian Zhang – Student Researcher – ian.zhang@utdallas.edu
Ryan Nelson – Student Researcher – Ryan.Nelson@UTDallas.edu
Samuel Ofiaza – Student Researcher – Samuel.Ofiaza@UTDallas.edu
Brian Ricks – Faculty Support – absolutefunk@utdallas.edu

Problem Mentors:

Brad Kline - National Security Agency - bjkline@uwe.nsa.gov
Drew Wilson - National Security Agency - aewilson@uwe.nsa.gov
Sam Osofsky - National Security Agency - stosofs@radium.ncsc.mil

Table of Contents

1. Project Summary	2
1.1 Top Level Research / Description	2
1.2 Keywords	2
1.3 Terminology	2
1.4 Team Member Responsibilities	2
1.5 Estimated Project Budget	2
1.6 Summary of Key Deliverables	2
2.0 Executive Summary	3
3.0 Motivation	3
4.0 Previous Work	4
4.1 Signal Modeling Using Recurrent Neural Networks [1]	4
4.2 1D Convolutional Neural Networks and Applications: A Survey [2]	4
4.3 Digital Signal Processing Using Deep Neural Networks [3]	5
5.0 Project Goals	6
5.1 Primary Goal	6
5.2 Specific Objectives	6
1. Data Processing Pipeline	6
2. Model Development and Optimization	6
3. Progressive Difficulty Testing	6
4. Ensemble Model Creation	6
5.3 Success Evaluation	6
5.4 Contingency Planning	6
6.0 Research Plan / Approach	7
6.1 Research Project Plan / Approach	7
6.2 Project Schedule	8
6.3 Resources Required / Budget	9
6.4 Progress to Date	9
6.4.1 CNN	9
7.0 Deliverables	15
8.0 Issues and Challenges	16
8.1 Issues and Challenges Anticipated	16
8.2 Issues and Challenges Encountered	16
9.0 Broader Impact	16
10. Investigator Biographies	17
11. Bibliography	18
Appendix A: Acronyms	18

1. Project Summary

1.1 Top Level Research / Description

The goal of this project is to develop an AI-driven pipeline capable of extracting sensitive data from unintended radio frequency emanations using machine learning techniques. Our team will investigate various neural network architectures to create a proof-of-concept character classifier that demonstrates the security vulnerabilities of RF side-channel attacks.

1.2 Keywords

Artificial Intelligence, Machine Learning, Radio Frequencies, Numpy/CSV, Neural Networks

1.3 Terminology

Neural Networks: It is a type of machine learning architecture that has many different nodes and layers that allow a model to “learn” through different patterns from them

Radio Frequency Emanations: these are radio frequency signals that get emitted as a byproduct of an electronic device performing some operations

1.4 Team Member Responsibilities

Adi	Neural Network Design and Implementation
Brigham	Neural Network Design and Implementation
Ryan	Data Preprocessing and Feature Extraction
Ian	Ensemble Model Design and Implementation
Sam	Model Testing and Evaluation

1.5 Estimated Project Budget

There are no anticipated costs related to this project.

1.6 Summary of Key Deliverables

The key deliverables include having a “working” model that is able to classify radio frequency emanations, corresponding charts/graphs that detail the performance of our model, and a report that encapsulates this (as well as our overall methodology for getting the data, training/testing the model, etc.).

2.0 Executive Summary

This project focuses on using AI/ML to map unintended radio frequency (RF) emanations into underlying data (e.g., bytes). The central research question is whether machine learning models can reliably identify data patterns from RF signals despite variations in recording conditions. This is important because by understanding what emanations correspond to sensitive data, we can better safeguard systems against potential interception and reconstruction by threat actors.

Our approach will explore an ensemble of neural networks—including multi-layer perceptrons, recurrent neural networks, and convolutional neural networks—for classifying RF emanations. Data preprocessing will emphasize extracting features most relevant for accurate classification, while we will also investigate robustness by considering multiple simultaneous recordings.

The expected outcome is a working model capable of accurately classifying RF emanations into their underlying data. Deliverables include reproducible code, experimental results with graphs, and a technical report. Ultimately, this project will demonstrate both the feasibility of using AI for reconstructing data from emanations and provide insights into mitigation strategies to protect sensitive electronic systems.

3.0 Motivation

Modern electronic systems emit radio frequency signals whenever they do an operation. While not important initially, there are tools that allow adversaries to be able to reconstruct the data/operations from these signals. This poses a threat to cybersecurity since this bypasses traditional encryption and software defenses by exploiting a physical layer of computation.

The significance of this project lies in its potential to both expose and mitigate these risks. Demonstrating that AI/ML models can successfully recover data from RF emanations underscores the urgency for developing countermeasures, such as shielding, emission-limiting design practices, or even expanding upon this reconstruction framework to flag potential leaks. This project would have significant effects regarding classified, financial, and/or personal information.

By focusing on approximating data based on RF emanations, this project addresses a concrete and underexplored problem: unintended electromagnetic leakage as a means for information compromise. The goals of classifying and reconstructing data from these signals directly support the broader aim of evaluating system vulnerabilities and guiding the development of effective countermeasures. In doing so, the project contributes not only to advancing the science of signal analysis and AI but also to strengthening the resilience of electronic systems in an increasingly interconnected world.

4.0 Previous Work

This section examines relevant prior work that informs the methodological approach for RF emanation-based character classification. The literature review focuses on three key areas: recurrent neural networks for temporal signal modeling, one-dimensional convolutional neural networks for signal classification, and deep learning applications in digital signal processing. These works demonstrate the potential for neural networks to automatically extract features from RF signals while handling complexities like noise, interference, and temporal dependencies.

4.1 Signal Modeling Using Recurrent Neural Networks [1]

In this article, the author discusses the application of recurrent neural networks (RNNs) for signal modeling tasks, which directly relates to the RF emanation analysis that will be conducted in this project. Agarwal emphasizes that traditional signal processing methods often struggle with complex, non-linear signal patterns. Deep learning fills the gap left by these methods and is an effective approach to signal classification and reconstruction tasks.

The paper demonstrates that RNNs are adept at capturing temporal dependencies in signal data due to their ability to maintain information across time steps. This is particularly true for Long Short-Term Memory (LSTM) networks. This is an important consideration when analyzing RF emanations where the temporal characteristics of the signal carry information about the underlying data being processed. The author shows that RNNs can effectively model both short-term fluctuations and long-term patterns in signal data, making them a strong consideration for reconstruction from RF emissions.

The article also discusses preprocessing techniques specific to signal data, including normalization methods and windowing approaches that preserve temporal relationships. For this project, this suggests that careful preprocessing of the complex-valued RF recordings will be essential, particularly in maintaining phase coherence across multiple simultaneous recordings. The paper concludes that while RNNs show promise for signal modeling, the choice of architecture should be guided by the specific temporal characteristics of the signal being analyzed.

4.2 1D Convolutional Neural Networks and Applications: A Survey [2]

Kiranyaz et al. provide a comprehensive survey of one-dimensional convolutional neural networks (1D CNNs) and their applications in signal processing and classification tasks. This work is particularly relevant to the RF emanation project as it addresses the use of CNNs for processing time-series data, which aligns with the temporal nature of RF signal recordings.

The authors demonstrate that 1D CNNs are particularly effective for signal classification tasks because they can automatically learn relevant features from raw signal data without requiring extensive manual feature engineering. This is significant for the RF emanation project, where the optimal RF features for character classification may not be immediately obvious. The survey

shows that 1D CNNs can capture local patterns in signals through their convolutional filters while using pooling operations to achieve translation invariance, which could help with robustness as the dataset increases in complexity.

The paper also discusses various architectural considerations for 1D CNNs, including filter sizes, depth, and the use of residual connections. For the RF emanation analysis, this suggests that experimentation with different CNN architectures will be necessary to find the optimal configuration for character classification. The authors emphasize that 1D CNNs often outperform traditional machine learning methods on signal classification tasks while requiring less preprocessing than methods that rely on hand-crafted features. The survey concludes that 1D CNNs represent a powerful tool for signal analysis applications, particularly when combined with other deep learning techniques.

4.3 Digital Signal Processing Using Deep Neural Networks [3]

This DOE/PNNL technical report explores the application of deep neural networks to digital signal processing tasks, providing insights directly applicable to the RF emanation analysis project. The report emphasizes that traditional signal processing techniques, while well-established, often struggle with complex, real-world signals that contain noise, interference, and non-linear characteristics commonly found in RF emanations.

The document discusses several key advantages of using deep learning for signal processing, including the ability to learn complex feature representations automatically and handle non-stationary signals effectively. This is particularly relevant for this project, where RF emanations may vary significantly based on environmental factors, distance from the source, and the specific hardware generating the emissions. The report demonstrates that neural networks can be trained to be robust to these variations while maintaining high classification accuracy.

The technical report also addresses the challenge of working with complex-valued signals, which is a challenge that will be addressed in this project. The authors discuss various approaches for handling complex signals in neural networks, including treating real and imaginary components separately or using specialized complex-valued neural network layers. For the character classification task, this suggests that careful consideration of how to represent the complex-valued RF data as input to the machine learning models will be essential in achieving the highest accuracy.

The report concludes that while deep neural networks show significant promise for signal processing applications, their success depends heavily on having sufficient training data and appropriate preprocessing techniques. This aligns with the project's progressive difficulty approach, where starting with simpler datasets allows for model development and validation before tackling more challenging RF emanation scenarios.

5.0 Project Goals

Our research aims to develop AI-driven methods for extracting sensitive information from unintended RF emanations. The following specific, measurable goals guide our work:

5.1 Primary Goal

Develop a reliable AI pipeline for character classification from RF emanations with >80% accuracy on the most challenging dataset tier, demonstrating the feasibility of data reconstruction attacks via electromagnetic side channels.

5.2 Specific Objectives

1. Data Processing Pipeline

- Process I/Q samples into multiple input formats (raw, magnitude/phase, spectrograms)
- Achieve >95% data preprocessing success rate
- Compare performance across different data representations

2. Model Development and Optimization

- Train and evaluate 3+ ML architectures (CNNs, RNNs, MLPs, ensembles)
- Achieve >60% baseline accuracy on intermediate difficulty datasets
- Optimize hyperparameters for top 3 performing models

3. Progressive Difficulty Testing

- Successfully classify characters across 3+ difficulty tiers
- Maintain <30% accuracy drop from easiest to hardest datasets
- Document performance degradation patterns

4. Ensemble Model Creation

- Combine top models into ensemble architecture
- Demonstrate consistent performance across multiple runs

5.3 Success Evaluation

Primary Metrics: Classification accuracy, precision/recall, F1-scores

Robustness Testing: Performance under varying noise conditions and recording environments

Practical Impact: Feasibility assessment and countermeasure recommendations

5.4 Contingency Planning

- If target accuracy is not achieved, we will:
- Analyze failure modes and model limitations
- Extract insights about RF signal recoverability patterns
- Document lessons learned for future research

Deliverables: Functional AI pipeline, comprehensive performance analysis, security recommendations, and technical documentation.

6.0 Research Plan / Approach

6.1 Research Project Plan / Approach

Two datasets containing labeled and unlabeled RF data will be provided by our sponsors. We will use the labeled dataset for supervised learning, and the unlabeled dataset for testing and future model evaluation. The labeled dataset will be split into subsets based on the degree of technical difficulty required to accurately classify the data in each subset.

The input data will be preprocessed and transformed into the following: raw I/Q samples, magnitude/phase samples, and spectrograms. Further feature extraction and data augmentation can be performed for more specialized models and techniques.

An iterative development approach will be used, where multiple models will be fed data sourced from the same subset/difficulty level. Starting from the subset with the lowest difficulty level, each model is trained and optimized to achieve a baseline level of accuracy. The idea is to slowly build up the capabilities of each model as newer, more difficult data is progressively introduced. After all models are optimized and compared, the best performing models will be used to create an ensemble model that aggregates what the others have learned, and will be used for final classification.

During the first portion of the semester, we will focus our attention on exploring as many machine learning techniques and models as time permits (e.g., feed forward NNs, CNNs, RNNs, clustering methods, transformers, pretrained models, traditional ML techniques).

For each technique or model, we aim to achieve the following:

1. Determine an effective data format suited to the technique
2. Perform hyperparameter tuning to optimize performance
3. Compare the resulting metrics of this technique with other techniques

Two weeks before the final report deliverable due date, we will create a ranking of the final versions of each model and select the top three best-performing candidates to be included in the development of a final ensemble model. We will extract each models' last representation of the data before classification (e.g., embeddings) and use them to train a final ensemble model that incorporates the learned features of the previous models. Some options for the ensemble model include Multi-Layer Perceptron, XGBoost, and LightGBM.

The last week before the final report deadline will be reserved to work exclusively on the report and presentation.

6.2 Project Schedule

Phase	Milestones	Deadline
Environment Setup	(1) Set up shared GitHub (2) Upload datasets (3) Configure local environments to use PyTorch/TensorFlow	Sep. 19, 2025
Iterative Model Development	(1) Select up to 5 ML models/techniques (2) Train and optimize each model using progressively more difficult datasets (3) Generate metrics from testing each model the most difficult dataset	Nov. 21, 2025
Ensemble Model Development	(1) Rank each model and select the top 3 models (2) Extract embeddings or equivalent from each model (3) Train and optimize the final ensemble model (4) Generate metrics from testing the model the most difficult dataset	Dec. 4, 2025
Report Preparation	(1) Document both individual and final model evaluation on both labeled and unlabeled data (2) Prepare and submit a report and presentation including all findings	Dec. 11, 2025

6.3 Resources Required / Budget

All resource-intensive machine learning tasks will be done locally on our personal machines, and all required software is available either free or through UTD student licenses.

Resource	Description	Cost
Model Training Hardware	Computers used to train, test, and validate models	\$0
Documentation Software	Microsoft Word, Google Docs	\$0
Presentation Software	Microsoft PowerPoint, Google Slides	\$0
Team Time Commitment	Time spent for project development and reporting	Priceless

7.0 Current Progress/Results

7.1 CNN

To classify individual characters from complex-valued RF emanation data, we designed a 1D Convolutional Neural Network (CNN) tailored to the structure of our I/Q signal samples. Each template was converted into a fixed-size input of shape (11, 2), where 11 corresponds to the padded temporal context window (head + character + tail), and 2 represents the I and Q components of the signal.

Preprocessing

We first parsed raw template files to extract the result field containing variable-length I/Q vectors. To unify these into a consistent input shape, we applied symmetric zero-padding to align all samples to 11. For templates with fewer than 5 head/tail padding, the missing context was padded with zeros. Each tensor was then standardized using the mean and standard deviation computed from the training set.

After normalization, the dataset was randomly shuffled and split into three disjoint subsets: 70% for training, 15% for validation, and 15% for test. The test set remained untouched until final evaluation.

Model Architecture

The model begins with an input layer accepting the I/Q waveform of shape (11, 2). The first convolutional block applies a Conv1D layer with 64 filters and a kernel size of 3, using the ReLU activation function and same padding to preserve the temporal resolution. This is followed by batch normalization to stabilize training and max pooling with a pool size of 2, reducing the sequence length by half.

The second block increases the model's depth by using 128 filters in the Conv1D layer with the same kernel size and activation. Again, batch normalization is applied for training stability, and max pooling reduces the sequence further. These two blocks extract progressively higher-level local patterns from the I/Q waveform.

In the third convolutional block, a Conv1D layer with 256 filters and kernel size 3 is used to capture even more abstract features. This is followed by another batch normalization layer. After this, we include a pointwise Conv1D layer with 512 filters and kernel size 1, which increases the representational capacity without changing the spatial dimension. A final batch normalization layer follows this.

To convert the feature maps into a fixed-size vector, we apply global average pooling, which aggregates the information across the temporal dimension, producing a compact and translation-invariant representation of the signal.

The resulting vector is passed through a dropout layer with rate 0.5 to reduce overfitting by randomly deactivating half of the units. A fully connected dense layer with 256 units and ReLU activation follows, which allows the model to learn high-level interactions among the features. Another dropout layer with rate 0.3 is applied for additional regularization.

Finally, the model concludes with a dense output layer with units equal to the number of character classes (128), using the softmax activation to produce a probability distribution over all possible character labels.

This architecture is designed to be lightweight yet expressive. The layered Conv1D blocks capture both local and global temporal dependencies in the waveform, while batch normalization and dropout ensure better generalization. The use of global average pooling makes the model robust to minor shifts or jitter in the input signal window.

Model Training

The Adam optimizer is an adaptive learning rate optimization algorithm that combines the advantages of AdaGrad and RMSProp. It dynamically adjusts the learning rate for each parameter during training based on first and second moments of gradients. This makes Adam particularly effective for problems like ours where the input features (I/Q signals) vary in scale and statistical characteristics.

We use the sparse categorical cross-entropy loss function because the output labels are integer-encoded (not one-hot encoded). This loss function measures the difference between the model's predicted probability distribution (from the softmax output) and the true class index, encouraging the model to assign higher probabilities to the correct class.

The batch size is set to 64, meaning the model processes 64 samples at a time before updating its weights, and training is allowed to run for a maximum of 100 epochs.

To prevent overfitting and improve convergence, two important callbacks are used during training:

1) EarlyStopping:

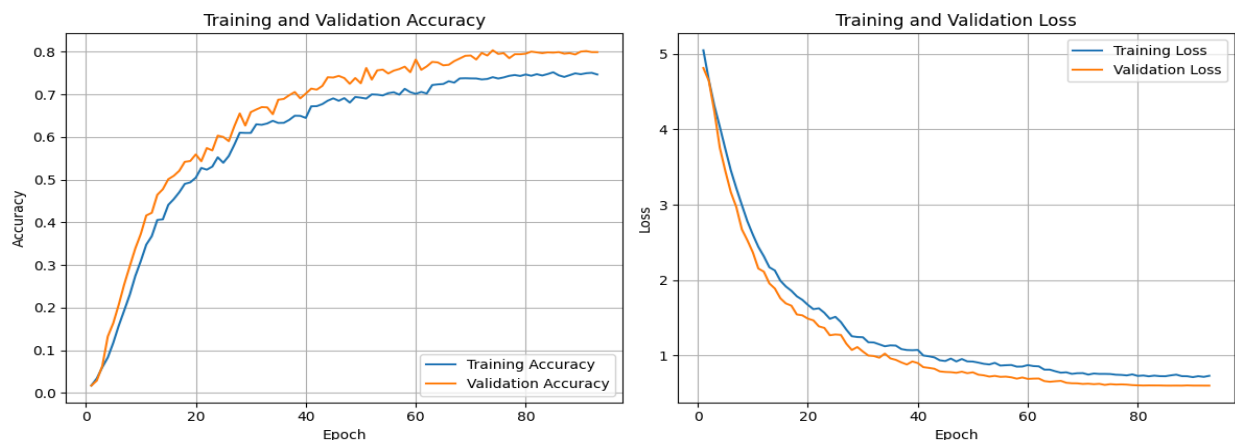
This callback monitors the validation loss and stops training automatically if it does not improve for 5 consecutive epochs (`patience=5`). Once triggered, it restores the model weights from the epoch with the best validation performance (`restore_best_weights=True`). This ensures that training halts at the point of optimal generalization, avoiding unnecessary epochs that could lead to overfitting.

2) ReduceLROnPlateau:

This callback monitors the validation loss (`monitor='val_loss'`) and automatically reduces the learning rate by a factor of 0.5 if the loss does not improve for 2 epochs (`patience=2`). Lowering the learning rate allows the optimizer to make smaller, more precise adjustments to the model parameters, often leading to better convergence and finer optimization near local minima.

Together, these two callbacks implement adaptive learning control—EarlyStopping prevents overfitting, while ReduceLROnPlateau fine-tunes the optimization process to improve validation performance.

Results





	precision	recall	f1-score	support
!	0.9	0.75	0.82	12.0
"	0.89	0.62	0.73	13.0
#	1.0	0.92	0.96	13.0
\$	0.8	0.92	0.86	13.0
%	0.77	0.83	0.8	12.0
&	0.67	0.46	0.55	13.0
'	0.78	0.54	0.64	13.0
(	0.92	0.85	0.88	13.0
)	1.0	0.69	0.82	13.0
*	0.92	1.0	0.96	12.0
+	1.0	0.75	0.86	12.0
,	1.0	0.75	0.86	12.0
-	0.62	0.67	0.64	12.0
.	0.85	0.85	0.85	13.0
/	0.61	0.85	0.71	13.0
0	0.77	0.83	0.8	12.0
1	0.79	0.85	0.81	13.0
2	0.91	0.77	0.83	13.0
3	0.43	0.75	0.55	12.0
4	0.82	0.75	0.78	12.0
5	0.91	0.77	0.83	13.0
6	0.71	0.77	0.74	13.0
7	0.8	0.67	0.73	12.0
8	0.92	0.92	0.92	13.0
9	1.0	0.67	0.8	12.0
:	0.77	0.83	0.8	12.0
;	0.71	1.0	0.83	12.0
<	1.0	1.0	1.0	12.0
=	1.0	0.77	0.87	13.0
>	1.0	0.62	0.76	13.0
?	0.82	0.75	0.78	12.0
@	0.9	0.75	0.82	12.0
A	0.75	0.92	0.83	13.0
ACK	0.89	0.67	0.76	12.0
B	0.64	0.54	0.58	13.0
BEL	0.68	1.0	0.81	13.0
BS	0.5	0.5	0.5	12.0
C	0.86	0.92	0.89	13.0
CAN	0.87	1.0	0.93	13.0
CR	0.92	0.92	0.92	13.0
D	0.77	0.83	0.8	12.0
DC1	0.8	0.92	0.86	13.0
DC2	0.56	0.83	0.67	12.0
DC3	1.0	0.85	0.92	13.0
DC4	0.8	0.67	0.73	12.0
DEL	0.83	0.42	0.56	12.0
DLE	0.67	0.92	0.77	13.0
E	1.0	0.75	0.86	12.0
EM	0.72	1.0	0.84	13.0
ENQ	0.75	0.69	0.72	13.0
EOT	1.0	0.92	0.96	13.0
ESC	1.0	0.85	0.92	13.0
ETB	0.93	1.0	0.96	13.0
ETX	1.0	1.0	1.0	13.0
F	0.77	0.77	0.77	13.0
FF	0.83	0.83	0.83	12.0
FS	0.86	0.5	0.63	12.0
G	0.81	1.0	0.9	13.0
GS	0.44	0.54	0.48	13.0
H	0.73	0.85	0.79	13.0
I	0.76	1.0	0.87	13.0
J	0.53	0.77	0.62	13.0
K	1.0	0.5	0.67	12.0
L	0.6	0.92	0.73	13.0
LF	0.91	0.83	0.87	12.0
M	0.57	0.67	0.62	12.0
N	0.85	0.92	0.88	12.0

NAK	0.82	0.75	0.78	12.0
NUL	0.62	1.0	0.76	13.0
O	1.0	0.75	0.86	12.0
P	0.9	0.75	0.82	12.0
Q	1.0	0.69	0.82	13.0
R	0.75	0.92	0.83	13.0
RS	0.75	0.92	0.83	13.0
S	1.0	0.77	0.87	13.0
SI	0.73	0.62	0.67	13.0
SO	0.67	0.92	0.77	13.0
SOH	0.75	0.5	0.6	12.0
SP	1.0	0.92	0.96	12.0
STX	0.8	0.92	0.86	13.0
SUB	0.61	0.92	0.73	12.0
SYN	0.85	0.92	0.88	12.0
T	0.92	0.92	0.92	13.0
TAB	0.86	0.46	0.6	13.0
U	1.0	1.0	1.0	12.0
US	0.72	1.0	0.84	13.0
V	1.0	0.77	0.87	13.0
VT	1.0	0.77	0.87	13.0
W	1.0	0.92	0.96	13.0
X	0.86	1.0	0.92	12.0
Y	0.8	0.62	0.7	13.0
Z	0.86	0.92	0.89	13.0
[	0.71	0.92	0.8	13.0
\	0.9	0.69	0.78	13.0
]	1.0	0.83	0.91	12.0
^	0.91	0.83	0.87	12.0
_	1.0	1.0	1.0	12.0
`	0.67	0.92	0.77	13.0
a	0.71	0.83	0.77	12.0
b	0.56	0.69	0.62	13.0
c	1.0	0.62	0.76	13.0
d	1.0	0.69	0.82	13.0
e	0.88	0.54	0.67	13.0
f	0.87	1.0	0.93	13.0
g	0.6	0.75	0.67	12.0
h	0.79	0.92	0.85	12.0
i	0.75	0.92	0.83	13.0
j	0.69	0.69	0.69	13.0
k	0.7	0.54	0.61	13.0
l	1.0	0.92	0.96	13.0
m	1.0	0.85	0.92	13.0
n	0.77	0.83	0.8	12.0
o	0.8	0.62	0.7	13.0
p	1.0	0.83	0.91	12.0
q	0.91	0.77	0.83	13.0
r	0.89	0.67	0.76	12.0
s	1.0	0.85	0.92	13.0
t	0.79	0.85	0.81	13.0
u	0.79	0.92	0.85	12.0
v	0.58	0.85	0.69	13.0
w	0.5	0.69	0.58	13.0
x	1.0	0.67	0.8	12.0
y	0.81	1.0	0.9	13.0
z	0.75	0.69	0.72	13.0
{	0.81	1.0	0.9	13.0
	0.8	0.67	0.73	12.0
}	1.0	0.33	0.5	12.0
~	0.67	0.77	0.71	13.0

The CNN model demonstrated strong overall performance in character recognition, with most ASCII characters achieving precision, recall, and F1-scores above 0.80. High-performing

characters such as C, EOT, ESC, P, TAB, VT, and several lowercase letters (l, p, t, x, y) reached near-perfect scores, indicating that the model effectively captures their distinguishing patterns.

However, several characters exhibited weaker performance. Symbols like &, }, and control characters such as DEL, DC2, and FS showed lower recall and F1-scores, suggesting difficulty in distinguishing these characters.

7.2 Hybrid LSTM + MLP

LSTMs (Long Short-Term Memory) are a type of RNNs that capture long term dependencies in sequential data. This makes sense since we are dealing with radio frequencies, which may/may not have dependencies based on the time elapsed. MLPs (Multi-Layer Perceptrons) are a type of ANN (Artificial Neural Network) that has many layers of neurons. This enables it to learn complex patterns, proving useful when looking at the metadata of the radio frequencies (signal head length, tail length, padding, etc). To get the best of both worlds, we go with a hybrid approach.

Preprocessing

Preprocessing is the same as what occurred in the CNN. However, the data is split up as 80% train and 20% test/validation.

Model Architecture

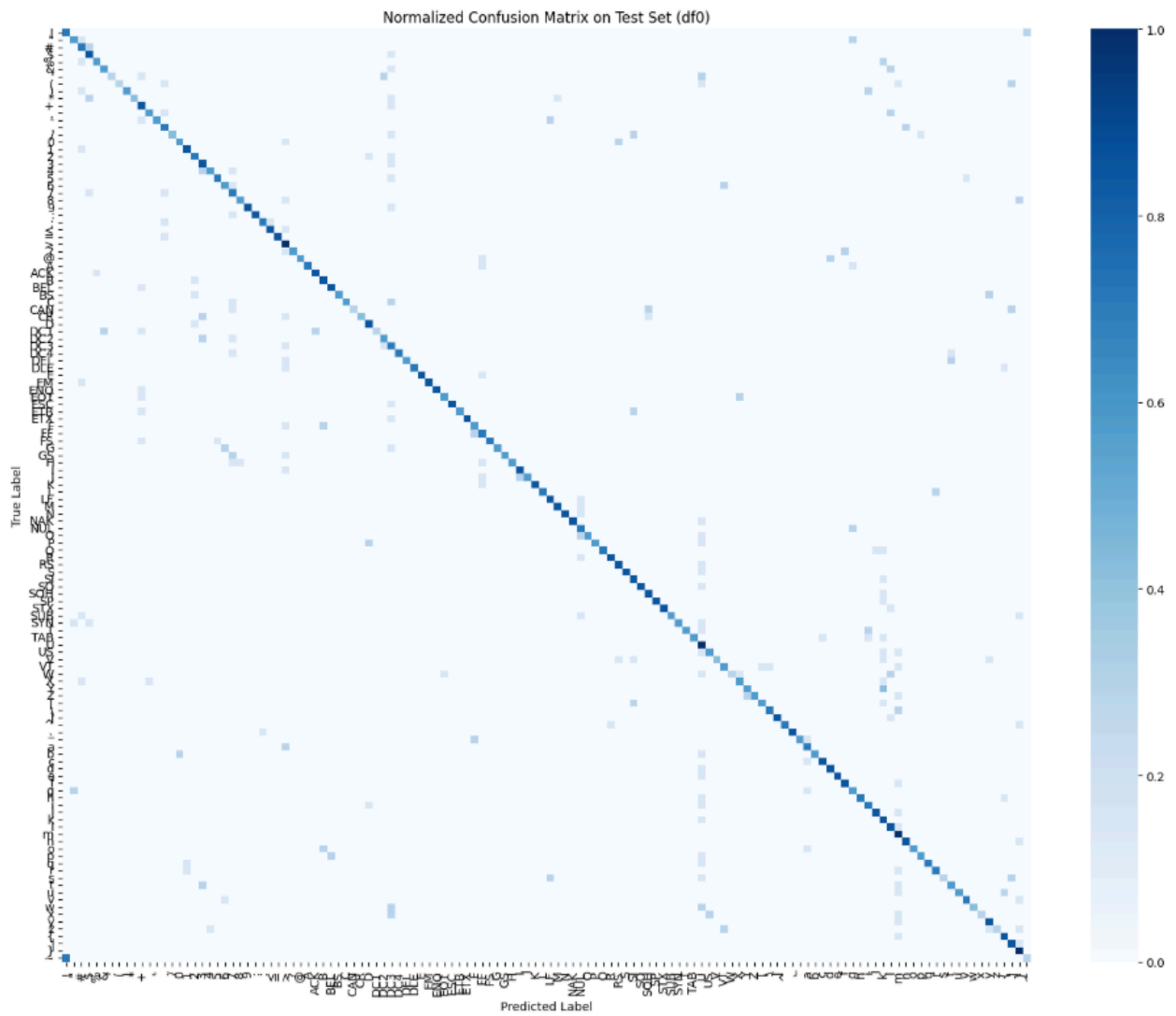
Similar as stated before, we go with a hybrid approach of using the LSTM for the sequential RF data and the MLP for the static metadata. For the LSTM, the input is the number of numerical features (20) and the output is an encoded output vector of 64 features derived from the output layer. For the MLP, the input is the number of static features (5), and the output is a 16-dimensional vector. Both are concatenated side by side to form a single combined feature vector, which is then passed into a classifier that makes the final prediction.

For the hyperparameter choices:

- Hidden size of LSTM is 64
- Number of layers for LSTM is 2
- Dropout of 0.3 and 0.5 for the MLP and the classifier (built in dropout for LSTM)
- An optimizer of Adam is used (most common)
- Learning rate of 0.001 and 200 epochs
- Loss function is cross entropy loss (suitable for classification tasks)

Results

The overall accuracy was around 68%.



	precision	recall	f1-score	support					
!	0.50	0.71	0.59	28	N	1.00	0.86	0.92	28
"	0.57	0.57	0.57	28	NAK	1.00	0.86	0.92	28
#	0.42	0.71	0.53	28	NUL	0.42	0.71	0.53	28
\$	0.50	0.86	0.63	28	O	1.00	0.57	0.73	28
%	0.80	0.57	0.67	28	P	1.00	0.57	0.73	28
&	0.67	0.57	0.62	28	Q	1.00	0.71	0.83	28
'	1.00	0.29	0.44	28	R	0.86	0.86	0.86	28
(	1.00	0.29	0.44	28	RS	0.67	0.86	0.75	28
)	1.00	0.57	0.73	28	S	1.00	0.86	0.92	28
*	1.00	0.43	0.60	28	SI	0.46	0.86	0.60	28
+	0.43	0.86	0.57	28	SO	1.00	0.86	0.92	28
,	0.80	0.57	0.67	28	SOH	0.67	0.86	0.75	28
-	1.00	0.57	0.73	28	SP	1.00	0.86	0.92	28
.	0.50	0.71	0.59	28	STX	1.00	0.86	0.92	28
/	1.00	0.43	0.60	28	SUB	1.00	0.57	0.73	28
0	0.67	0.57	0.62	28	SYN	1.00	0.57	0.73	28
1	0.75	0.86	0.80	28	T	1.00	0.57	0.73	28
2	0.62	0.71	0.67	28	TAB	1.00	0.57	0.73	28
3	0.43	0.86	0.57	28	U	0.23	1.00	0.37	28
4	0.80	0.57	0.67	28	US	0.67	0.57	0.62	28
5	0.83	0.71	0.77	28	V	1.00	0.43	0.60	28
6	0.57	0.57	0.57	28	VT	0.50	0.57	0.53	28
7	0.33	0.71	0.45	28	W	1.00	0.29	0.44	28
8	0.80	0.57	0.67	28	X	0.57	0.57	0.57	28
9	1.00	0.86	0.92	28	Y	0.67	0.57	0.62	28
:	1.00	0.86	0.92	28	Z	1.00	0.57	0.73	28
;	0.83	0.71	0.77	28	[	0.80	0.57	0.67	28
<	0.86	0.86	0.86	28	\	0.83	0.71	0.77	28
=	1.00	0.86	0.92	28	]	1.00	0.86	0.92	28
>	0.35	1.00	0.52	28	^	1.00	0.71	0.83	28
?	1.00	0.57	0.73	28	_	1.00	0.86	0.92	28
@	1.00	0.57	0.73	28	`	1.00	0.57	0.73	28
A	1.00	0.71	0.83	28	a	0.56	0.71	0.62	28
ACK	0.75	0.86	0.80	28	b	1.00	0.57	0.73	28
B	0.60	0.86	0.71	28	c	0.86	0.86	0.86	28
BEL	0.75	0.86	0.80	28	d	0.75	0.86	0.80	28
BS	1.00	0.57	0.73	28	e	1.00	0.86	0.92	28
C	1.00	0.57	0.73	28	f	0.75	0.86	0.80	28
CAN	1.00	0.29	0.44	28	g	0.44	0.57	0.50	28
CR	1.00	0.43	0.60	28	h	1.00	0.71	0.83	28
D	0.60	0.86	0.71	28	i	0.50	0.71	0.59	28
DC1	1.00	0.29	0.44	28	j	0.86	0.86	0.86	28
DC2	0.57	0.57	0.57	28	k	0.30	0.86	0.44	28
DC3	0.22	0.71	0.33	28	l	0.43	0.86	0.57	28
DC4	1.00	0.71	0.83	28	m	0.32	1.00	0.48	28
DEL	1.00	0.57	0.73	28	n	0.75	0.86	0.80	28
DLE	1.00	0.71	0.83	28	o	1.00	0.57	0.73	28
E	1.00	0.86	0.92	28	p	0.80	0.57	0.67	28
EM	1.00	0.86	0.92	28	q	1.00	0.71	0.83	28
ENQ	1.00	0.86	0.92	28	r	0.71	0.71	0.71	28
EOT	0.80	0.57	0.67	28	s	1.00	0.29	0.44	28
ESC	1.00	0.86	0.92	28	t	0.57	0.57	0.57	28
ETB	1.00	0.57	0.73	28	u	1.00	0.57	0.73	28
ETX	1.00	0.86	0.92	28	v	0.83	0.71	0.77	28
F	0.50	0.57	0.53	28	w	1.00	0.43	0.60	28
FF	0.45	0.71	0.56	28	x	1.00	0.29	0.44	28
FS	1.00	0.71	0.83	28	y	0.60	0.86	0.71	28
G	1.00	0.57	0.73	28	z	1.00	0.29	0.44	28
GS	1.00	0.57	0.73	28	{	0.60	0.86	0.71	28
H	1.00	0.57	0.73	28		0.50	0.86	0.63	28
I	0.75	0.86	0.80	28	}	0.47	1.00	0.64	28
J	1.00	0.57	0.73	28	~	0.50	0.29	0.36	28
K	1.00	0.86	0.92	28					
L	1.00	0.71	0.83	28					
LF	0.60	0.86	0.71	28					
M	0.86	0.86	0.86	28					
N	1.00	0.86	0.92	28					
NAK	1.00	0.86	0.92	28					

7.3 MLP

We trained and tested a Multilayer Perceptron (MLP) / Feed-forward neural network as a baseline. The accuracy of the model reached 83% before we moved on to other, more complex models.

Preprocessing

The template dataset was aggregated into a single Polars dataframe containing 10,752 samples, all labeled as 128 unique ASCII characters. All samples were padded with zeroes if their lengths were less than the maximum length of 22 elements.

Architecture

The MLP contains 504,448 total parameters. The layers of the model are described below.

- 22 neurons, ReLU activation
- 20% dropout
- 512 neurons, ReLU activation
- 20% dropout
- 256 neurons, ReLU activation
- 20% dropout
- 128 neurons

Model Training

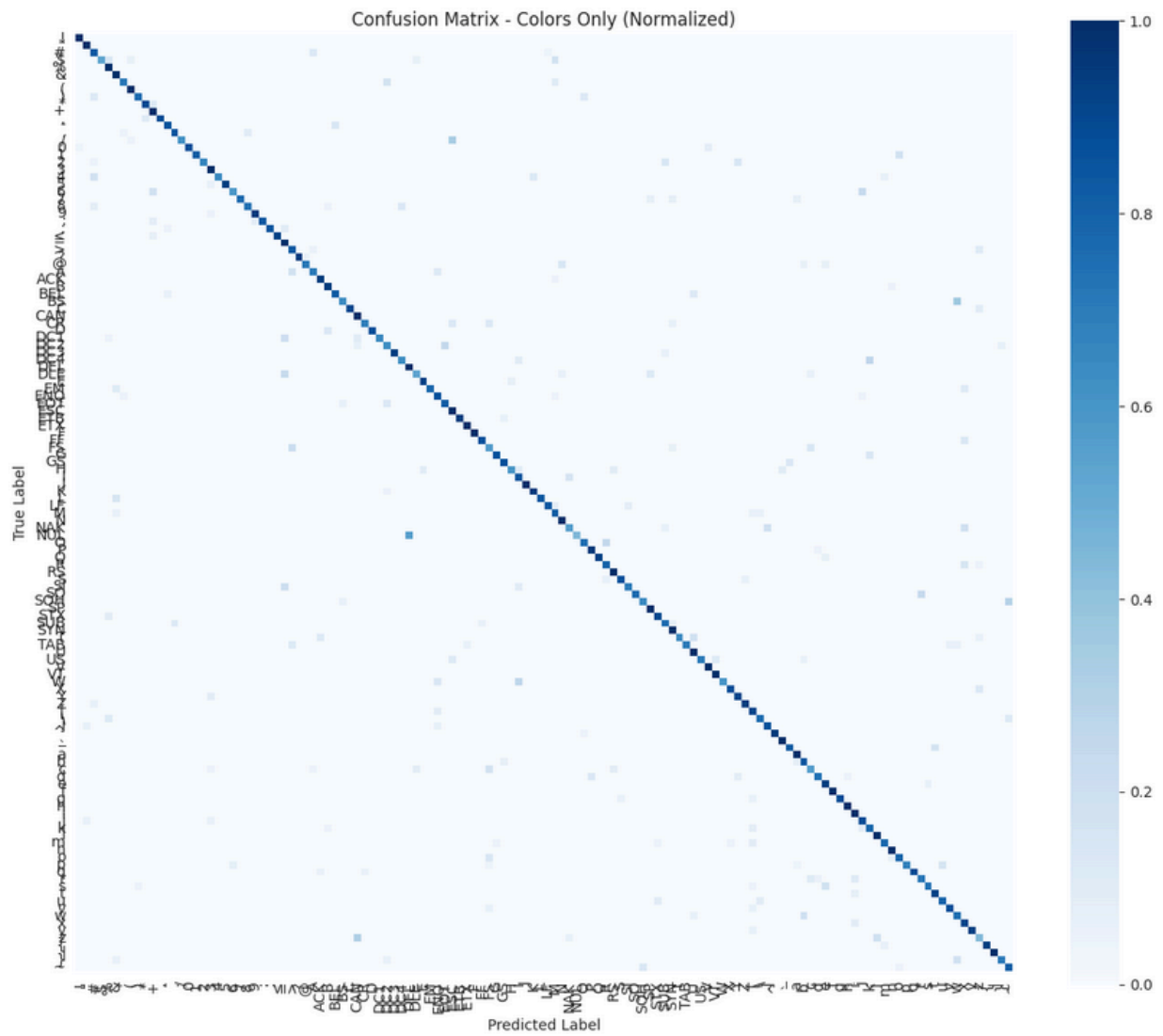
The hyperparameters used during model training are described below.

- 150 epochs
- Batch size of 8
- Learning rate of 0.001
- Optimizer: PyTorch's ReduceLROnPlateau
 - Dynamically decreases the learning rate by 50% after 5 epochs of plateaued accuracy occur
 - Minimum learning rate was set to 0.000001

Model checkpoints were saved during training.

Results

	precision	recall	f1-score
!	0.928571	1.000000	0.962963
"	0.900000	1.000000	0.947368
#	0.689655	0.833333	0.754717
\$	1.000000	0.535714	0.697674
%	0.619048	1.000000	0.764706
...	...	...	...
}	0.923077	0.705882	0.800000
~	0.720000	0.818182	0.765957
accuracy	0.830233	0.830233	0.830233
macro avg	0.849612	0.839277	0.832103
weighted avg	0.855835	0.830233	0.830592



8.0 Deliverables

Item	Due Date
Proposal	Sep. 12, 2025
Processed Datasets	Sep. 20, 2025
Progress Report	Oct. 24, 2025
Trained Selected Models	Nov. 21, 2025
Trained Ensemble Models	Dec. 4, 2025
Confusion Matrix	Dec. 11, 2025
Code	Dec. 11, 2025
Final Report	Dec. 11, 2025

9.0 Issues and Challenges

9.1 Issues and Challenges Anticipated

Several technical and logistical challenges are anticipated throughout the project.

- **Signal Noise and Interference:** Real-world RF emanations often include high levels of background noise and interference. Differentiating meaningful signal components from noise will be essential for accurate classification.
- **Data Labels:** The dataset contains both labeled and unlabeled RF recordings. While the labeled data supports supervised learning, the unlabeled data might present challenges. This may require manual inspection and alignment with known labeled patterns. Mistakes in the labeling stage could propagate through the training pipeline and degrade model performance.
- **RF Data Representations:** RF data is typically captured as I/Q samples (In-phase and Quadrature), meaning that each sample is a complex number. However, most conventional neural networks are designed to process real-world inputs. To use the data effectively, we must choose appropriate transformation methods to transform it into inputs for deep learning models.
- **Model Training:** Different model types (CNNs, RNNs, transformers) may require different learning rates, optimizers, and regularization techniques. Ensuring convergence to optimal or near-optimal solutions will require careful hyperparameter tuning, experimentation with batch sizes and optimizers, and consistent use of validation data to monitor for underfitting or overfitting. Additionally, ensembling models may require alignment of their internal feature representations to maximize combined performance.
- **Ensemble Complexity:** Creating an ensemble of different models introduces challenges. Embedding representations may need to be aligned or standardized before creating the ensemble. Experiments need to be performed to avoid performance degradation.

9.2 Issues and Challenges Encountered

The main roadblock encountered thus far is the inability to communicate with the project sponsors as a result of the government shutdown. We are able to continue with the project by addressing the sequences, though we cannot thoroughly test our template results without realistically noisy data. For now, we will base our model performance on the clean data initially provided by the sponsors. We may have to apply our own noise to the data, though that may be an unrealistic application as we do not have the thorough background to understand what realistic noise would look like.

10.0 Broader Impact

This project contributes meaningfully to the broader goals of system security, applied machine learning, and interdisciplinary research by demonstrating a practical framework for extracting information from unintended RF emanations. As machine learning becomes increasingly

integrated into security workflows, the ability to recover data from low-level emissions illustrates a new class of vulnerabilities that bypass traditional software-based protections. This work provides an experiment for assessing risk and testing mitigation strategies.

The findings may support the development of physical-layer defenses, such as emission suppression, shielding techniques, or architectural adjustments that reduce the recoverability of sensitive information. In addition, the model evaluation process—including performance comparisons across architectures and difficulty tiers—may provide insight into which signals or conditions are most susceptible to reconstruction.

The technical outputs of this project, including our code pipelines and trained models, will be valuable to both academic and applied research efforts in machine learning, side-channel analysis, and signal processing.

From an educational perspective, the project provides team members with hands-on experience in integrating deep learning with real-world RF data. Through iterative experimentation, the team will gain a comprehensive understanding of both signal-level phenomena and modern AI workflows. This hands-on process reinforces interdisciplinary skills in AI, cybersecurity, and signal processing, and prepares us for advanced work in secure systems and applied machine learning.

Finally, the project supports the broader goal of designing secure-by-default systems in an increasingly wireless and connected world. By identifying how and when data can leak through physical channels, and by demonstrating tools to detect or exploit such leaks, this work contributes to building more resilient infrastructure in critical sectors ranging from communications to defense.

11. Investigator Biographies

Adithya Chintala is a Scholarship for Service Student that is pursuing a master's in computer science and specializing in cybersecurity, with interests in combining AI with cybersecurity. He is currently working on research with a professor dealing with anomaly detection and has done personal projects such as exploiting a gun detection system based on a CNN (fgsm attack) and classifying spam email. This summer, Adithya interned at Idaho National Laboratory, where he worked on 2 projects: one which deals with using AI/ML techniques to speed up the reverse engineering process, and the other one deals with trying to detect hallucinations of a transformer on a token-by-token basis.

Brigham Thornock is in the final year of his master's in computer science with a focus in cybersecurity. His main areas of study are machine learning and offensive security. During the previous summer, he completed an internship at the Institute for Defense Analyses where he utilized large language models to identify metrics in government acquisition documents and assisted in a project identifying adversarial satellite image data. He is currently working towards completing the Offensive Security Certified Professional certification and looks to pursue a passion in breaking AI for a living.

Ryan Nelson brings relevant hardware security expertise to this project through his ownership and operation of ChipWhisperer platforms and completion of comprehensive side-channel attack tutorials. His background in implementing wireless networks in the development of an RF chain application for amateur satellites. With coursework in machine learning and programming experience in Python, Nelson possesses the foundational knowledge needed for deep learning applications, though he seeks to expand his knowledge specifically in neural network architectures for signal processing applications.

Ian Zhang is in the last semester of his Master's Degree in Computer Science in the track of AI/ML. He previously worked at Toyota as a Crypto Operations Intern, where he has established a PKI information visualization pipeline within Toyota's AWS infrastructure. At UTD, he has completed coursework in Artificial Intelligence and Machine Learning, further strengthening his technical expertise in AI/ML. This summer, he was a Software Engineer intern at F5 Networks, where he has worked with the team to improve the code security. He is interested in exploring the applicability of using machine learning models for real-world problems. He also seeks to gain more hands-on experience with state-of-art deep learning models.

Sam Ofiaza graduated with a B.S. in Computer Science with a focus on AI/ML and is currently working on a M.S. in Computer Science on the Cyber Security track. He is highly proficient with Python, Tensorflow, and PyTorch, as well the agentic AI frameworks LlamaIndex and LangChain. He has shown interest in combining his knowledge of AI and cybersecurity through a personal project exploring the effectiveness of various white-box adversarial attacks against a CNN-based image classifier. Some implemented attacks include a pixel noise loss maximizer, an implementation of Fast Gradient Sign Method (FGSM), model weight tampering, misclassification by watermarking, and an adversarial GAN.

12. Bibliography

- [1] Agarwal, A. (2019, June 11). Signal Modeling Using Recurrent Neural Networks. Retrieved September 12, 2025, from Medium website:
<https://pub.towardsai.net/signal-modelling-using-recurrent-neural-networks-d832f0c50503>
- [2] Kiranyaz, S. (2021). 1D convolutional neural networks and applications: A survey. *Mechanical Systems and Signal Processing*, 151, 107398.
<https://doi.org/10.1016/j.ymssp.2020.107398>
- [3] Shevitski, B., Watkins, Y., Man, N., & Girard, M. (2023). Digital Signal Processing Using Deep Neural Networks. *OSTI OAI (U.S. Department of Energy Office of Scientific and Technical Information)*. <https://doi.org/10.2172/1984848>

Appendix A: Acronyms

AI – Artificial Intelligence
ML – Machine Learning

RF – Radio Frequency
NN – Neural Network
CNN – Convolutional Neural Network
RNN – Recurrent Neural Network
LSTM – Long Short-Term Memory
I/Q – In-phase / Quadrature
DOE – Department of Energy
PNNL – Pacific Northwest National Laboratory
UTD – The University of Texas at Dallas
FGSM – Fast Gradient Sign Method
GAN – Generative Adversarial Network
AWS – Amazon Web Services
PKI – Public Key Infrastructure